Chapter 2. The Value of Reliability

By Ben McCormack, Brian Kudzia, David Huska, and Jose Andrade
with Garrett Holthaus

The smooth and consistent operation of technology services isn't discretionary; it's a critical factor for business success. This chapter discusses Site Reliability Engineering from a perspective that often isn't considered: as a business *investment* for the organization. While there are certainly business costs associated with implementing SRE, the measurable return on that investment can be substantial. We'll examine how SRE principles—such as managing risk, setting service performance goals, reducing repetitive tasks, improving system visibility, and automating processes—not only prevent expensive service disruptions but also drive efficiency and foster innovation throughout the organization.

Why SRE Is Right for Google

Traditionally, engineering teams were organized by separating software development from operations. This usually resulted in suboptimal outcomes because there was poor alignment between operational and software teams to balance stability and feature velocity. This organizational structure led to code being "thrown over the wall" and both teams blaming each other when failures occurred. Additionally, operations teams often fell into the trap of having humans doing repetitive tasks over and over again, which is both unrewarding for the individuals and inefficient for the business.

Over the past two decades, practices like DevOps and SRE have emerged to improve this situation. We believe these are superior approaches to developing and running systems compared to traditional operations teams.

At Google—a company rooted in a strong engineering culture—we view the challenge of running systems at scale through a software-first lens. Solving large-scale operational challenges with engineering rigor and automation is central to our philosophy.

It's worth acknowledging that not every system at Google is under the management of an SRE team; some are run and managed by the Software Engineering (SWE) teams (see Chapter 7). When it comes to mission-critical services, however, we use an SRE approach. We'll return to how and why SRE is right for Google throughout this book, but to sum it up succinctly, it is because SRE treats operational management as a software problem rather than a human one. Historically, managing growth meant hiring more people to handle an ever-growing list of manual, repetitive tasks. SRE recognizes that the engineering of reliability, availability, and scalability at scale is a distinct specialization, which, when done well, allows the organization to move past these limitations.

Furthermore, SRE provides a rewarding career path for engineers interested in reliability and scalability. Its focus on software engineering naturally leads to a high level of automation by software rather than humans needing to manually intervene. This fosters deep expertise in systems design and helps engineers understand how to optimize large-scale systems and their architecture. While other approaches leave individual teams to "self-select" less toil, which can lead to burned-out software teams, SRE specifically recognizes, measures, and manages toil (see Chapter 13). (SRE defines toil as work that is manual, repetitive, automatable, tactical, and often having limited enduring value—put simply, it's work that is unrewarding for humans and better served by a software-based solution.)

Optimization is another key reason why SRE is such a strong part of the organizational strategy at Google. SRE has allowed us to centralize key operational functions and standardize tools across teams. This shared foundation promotes efficiency and enables all teams to benefit from best practices developed and refined by other SRE teams.

NOTE

To learn more about SRE's role in the design, deployment, and maintenance of critical large-scale systems, listen to the DNS or Maglev episodes of *Prodcast*, Google's SRE Podcast.

The Value of SRE

Adopting SRE as an approach for the ongoing management of technical environments is a large commitment. For many organizations, it requires changing how work is done, which leads to changes to team structures and practices. Before undertaking such a large shift, it's important to consider the diverse stakeholders you'll need to engage with and to clearly understand the value of SRE. These stakeholders range from your customers to

your chief financial officer (CFO) and include all of the other internal teams that interface with reliability-focused teams (see also Chapter 7).

SRE's engineering-driven approach and focus on efficiency enable organizations to scale services and their supporting operations such that costs grow more slowly than the services themselves. We call this a *sublinear scaling* model. Teams begin to adopt core SRE tactics, such as protecting error budgets, which often leads to an intrinsic regulation of service reliability rather than one imposed from outside the team. Over time, this helps reduce operational toil, which frees up time to work on the next big objective. As teams learn and mature, this can reduce people management toil as well. Leadership and engineering can then spend less time dealing with recurring issues that arise from repetitive work and instead focus on more valuable tasks to move the organization forward.

Let's dig a bit deeper into the key differences between a classic ops team and SRE. An ops team is generally incentivized primarily by stability, often at the expense of change velocity. SREs, on the other hand, use error budgets to balance reliability with feature investments, aligning both SRE and development teams on common goals. SREs own product success and help break down the proverbial walls that might exist between traditional business, operations, and development teams. While ops teams historically focused on the sysadmin skill set, SREs are additionally software engineers who apply engineering principles to operational problems. This tends to lead to strong collaborative relationships between SRE and development teams, with shared ownership and mutual goals.

In the remainder of this chapter, we'll discuss how the value realized by SRE accrues to the different stakeholders within an organization, and how this value tends to differ between organizations at different stages of development.

To Customers

Customers want a system that is not just reliable to use but that evolves and is updated to meet their current and future needs. The key SRE principles of *embracing risk* while *balancing risk against business objectives* provide a mature approach to managing these often conflicting needs.

As engineers, we can sometimes get lost in exact definitions: Is this a service level agreement (SLA) or a service level objective (SLO)? Do we operate primarily with SRE or DevOps principles?

While defining the specifics is helpful, it sometimes distracts us from the most important thing for any team responsible for running and delivering a production service: providing a service to others. It is in this capacity that SRE can play an integral role in balancing the features your users want with system stability.

FOCUS ON THE USER AND ALL ELSE WILL FOLLOW

In 2001, three years after being incorporated, Google was going through hypergrowth. Larry Page and Sergey Brin wrote an engineering-first rulebook to anchor the company's culture and guide decisions during the intense organizational scaling. The following was, and still is, rule number 1:

> *Since the beginning, we've focused on providing the best user experience possible. Whether we're designing a new internet browser or a new tweak to the look of the homepage, we take great care to ensure that they will ultimately serve you, rather than our own internal goal or bottom line. Our homepage interface is clear and simple, and pages load instantly. Placement in search results is never sold to anyone, and advertising is not only clearly marked as such, it offers relevant content and is not distracting. And when we build new tools and applications, we believe they should work so well you don't have to consider how they might have been designed differently.*
>
> —Google, "Ten Things We Know to Be True" (emphasis added)

Meeting this goal starts with the availability of an application or service—if an application is unavailable, then it is not able to perform its function and is not serving its customers or users. Other qualities of a reliable service, such as operating quickly and correctly, likewise affect users' perception of value.

This means the reliability of the service is not an optional feature or a cost to be reduced as much as possible. Reliability is a critical feature and should be treated as such. It doesn't matter what other features or capabilities your product has: when the system is down, users can't use them! Broadly speaking, reliability is the core feature that everything is built upon.

Running a reliable system should never be confused with trying to achieve 100% uptime, which is almost always the wrong target. A key function of SRE is to define and deliver an appropriate level of reliability for each system. Extreme reliability comes at a cost: maximizing stability limits how fast new features can be developed and how quickly products can be delivered to users. It also dramatically increases their cost,

which in turn reduces the number of features a team can afford to offer. SRE seeks to balance the risk of poor reliability with the goals of rapid innovation and efficient service operations, so that users' overall happiness—with features, service, and performance—is optimized.

The customers of established businesses typically have expectations about the reliability of the services they offer. Even without having a specific SLO in mind, customers generally expect services to be available when needed and to operate smoothly and correctly. When they don't, it results in measurable dissatisfaction reflected by instruments like Customer Satisfaction (CSAT) and Net Promoter Scores, and of course customer spend. All the SRE practices outlined in this book will support the high reliability expected by customers, and because they're so important, we've dedicated an entire chapter (Chapter 8) to SLOs. There, we cover how to define good SLOs to identify and quantify customer expectations and make them central to the engineering process.

To the CFO

Cost within a business is a carefully managed and controlled activity. In a growing business, the CFO aims to increase expenses at a slower rate than top-line revenue. In a steady-state business, the focus shifts to optimizing cost over time.

Central to SRE philosophy is that machines are fast and inexpensive, while people are slower but far more valuable because of their creativity and problem-solving abilities. SRE's focus on automating repetitive manual tasks and streamlining tools and processes is key to supporting scale far in excess of the cost of achieving it—that is, sublinear scaling.

Software is one of the most powerful financial leverage tools we have. Code that is well designed, maintained, and deployed can be run again and again with minimal incremental cost. The key SRE principles of *eliminating toil* and *automation* reduce the cost to run and manage technical systems. Used correctly, this delivers strategic financial value to organizations by enabling sublinear scaling, which is crucial for any business wishing to grow sustainably.

The centralization of SRE services at Google has also allowed all teams to benefit from improvements in services used by multiple applications. Load balancing is a great example of this, with the vast majority of Google services that people know and use—Search, Maps, YouTube, etc.—all benefiting from a single global Load Balancing platform that is run and managed by SRE. To put it simply, efficiency gains are shared by everyone.

To Other Internal Technical Teams

SRE teams don't function in isolation within an organization; they partner with many other teams, including product, security, customer service, and more (see Chapter 7).

The implementation of objective measures of success, such as SLOs, can better align product development and infrastructure teams. SRE often works with product owners to turn business goals into explicit reliability objectives, advising on the risks and viability of different options when balancing requirements and trade-offs. The DevOps Research and Assessment (DORA) report from 2021 found evidence that "teams who excel at modern operational practices are 1.4 times more likely to report greater software delivery and operational (SDO) performance, and 1.8 times more likely to report better business outcomes."[1]

With enhanced communication, organizations can better understand business objectives and provide more realistic and achievable outcomes. This can not only lead to faster velocity, which keeps your developers happy, but also help establish better alignment with your business goals and—ultimately—increase customer happiness. It can also reduce risk and outage severity, which keeps your operators happy. In turn, achieving your reliability goals and desired outcomes helps shape the business and keep it on track. You can think of SRE as a way to unite your entire organization on the principle of reliability.

Better communication and a clearer focus on production outcomes can lead to gains in efficiency and efficacy. SRE provides deep production expertise—more than development teams would reasonably be able to build up—which helps make those teams more successful. They don't need to worry so much about production and how to hit their reliability targets, which reduces cognitive load and allows them to use their time more effectively. Having more room to focus on business logic allows them to deliver higher-quality products and meet more rigorous standards with less toil.

To Organizational Culture

One of the most important components of SRE is its culture, which exists within the broader organizational culture. SRE culture tends to influence that broader culture positively along measures such as how people perform at their jobs and the tenure employees have at the organization (see Chapter 4).

The decision to implement SRE in an organization *will* influence the overall culture. Certain components of SRE practice, such as emergency response, observability, change management, capacity planning, and cross-team alignment, might be new to the organi-

zation. And the influence extends beyond engineering. For example, it's important for sales and marketing to communicate when planned campaigns may impact the number of users, as significantly increased utilization often requires additional compute power.

Creating an environment where psychological safety is prioritized is required to implement SRE properly. For example, holding blameless retrospectives will empower team members to learn from incidents so the next time something similar arises, they can fix the issue before it becomes a major problem. This mindset will bring benefits to the organization, and over time, the reliability of the systems will improve. However, those decisions cannot be made without full leadership buy-in.

An assessment of the existing organizational culture can be helpful when deciding to implement SRE—especially if approached with an open mind and excitement about the opportunities. Once implemented, the culture of SRE will continue to bring benefits in the form of improved communication, faster decision making, clear rules, and improved reliability of systems, which is the main goal overall.

Leaders who understand the value of SRE, are supportive of engineering decisions based on data, and are able to influence teams not directly under their control are essential for the successful implementation and maintenance of a culture of reliability.

To Business Growth

SRE directly contributes to business growth by ensuring the reliability of revenue-generating services. Scalable systems and agile operations are essential to supporting rapid business growth even in unexpected or extraordinary circumstances. Consider the example of Google Meet during the COVID-19 pandemic. Video conferencing became a vital tool to help users work and stay connected. The volume of video calls handled by Meet exploded during the early stages of the lockdown. The resulting increase in needed resources was unplanned, but Google was able to accommodate its users thanks in large part to the reliability strategy that underlies its products. This was a big win for both Google and users of Meet, and it illustrates that a ready reliability strategy makes it possible to enhance critical time-to-market efforts through accelerated development and deployment processes. This can be a significant competitive advantage (see "Case Study: Throwing the Plan out the Window (Google Meet During COVID-19)").

This example illustrates SRE's core value: ensuring critical systems remain reliable and usable, even under pressure. For customer-facing services, success can be measured in happy customers who continue to use and even recommend the service. For internal ap-

plications that might not directly generate revenue, reliability is still crucial if the service is essential to core business operations.

SRE's reliability targets are directly correlated to the expectations of users. Therefore, it's important to understand the cost of not meeting these expectations in terms of lost revenue, reputation, etc. Having a firm understanding of users' expectations allows SREs to put the correct amount of effort into assuring the reliability of the system, and it can also be used as a data point for SRE to prove its own value. Defining SLOs will help your teams translate these system expectations into meaningful workflows (see Chapter 8).

Within the SRE Team

One of the most important things to consider is the impact that poor service quality has on employee and team morale. Being woken up in the middle of the night for the third time in a week is not fun for anyone; it has a real human impact on both individuals and teams. In SRE culture, people are considered the most important resource. When systems fail, the burden of remedying the problem falls on a small set of employees. Having to manually restart a process for the tenth time this week is another simple example of toil that can be a significant drain on a team. An important part of SRE is measuring and quantifying toil. Once identified, a significant portion of an SRE's time is spent automating it away (as mentioned previously, at Google, we aim to dedicate *at least* 50% of SRE time to engineering work; for more details, see "On-Call Considerations"). Reducing toil reduces overhead and repeated mistakes and improves team culture, leading to less burnout, better morale, and higher productivity (see Chapter 4).

At Google, SRE can push back on a development team if its application consistently fails to meet SLOs. If incident alerts (or "pages"[2]) are going to the on-call engineer so often that it affects the team's health and operation, the SREs can hand the "pager" back to the development team. This is a powerful lever to align development and operations on agreed-upon levels of service reliability.

The Cost of Unreliable Systems

Quantifying the value of problems that don't happen can be difficult. How many power outages and vehicle breakdowns did you *not* experience last year, and what was their aggregate cost that you *didn't* incur? The cost of degraded reliability varies by industry and by the size and maturity of an organization, but we can state general truths about it.

Reliability is crucial because it reflects user expectations for a service—a service is perceived as reliable only if it meets those expectations. Failure to do so breaks trust, which is fragile and difficult to regain once lost. Meanwhile, inside an organization, even a moderate outage can drive feature development to zero as engineers focus on mitigation and repair. These two consequences alone will place an enterprise at a competitive disadvantage. Think of this section as a study of the cost of *not* having a reliable system.

Direct Costs

Reliability failures have direct acute and chronic financial consequences for a business: lost revenue due to goods and services that aren't sold, additional expenditures to fix the problem, damage to reputation, etc. These vary by industry, business, and potentially even time of year or day.

What's more, in a competitive environment, customers have a choice, and if your service is unavailable or not performing as expected, they may simply go to a competitor. You are likely to lose not only the sale at that point in time but also future business from those customers. This is referred to as *churn*.

NOTE

Google research shows that 53% of visitors will abandon a task if they have to wait longer than three seconds for a page to load.

Opportunity Costs

Not investing in reliability means that engineering resources may be spent on reactive work fixing outages and incidents rather than developing new features or operational tools. This hinders proactive improvement and leads to a culture of reactionary work.

If you are providing an internal service to your business and it is unavailable, there may be hundreds or thousands of employees who are unable to do their work.

THE HIGH COST OF DOWNTIME: A REAL-WORLD EXAMPLE

A Googler who used to run and manage an enterprise resource planning (ERP)/finance system at another company relates this story:

> *One day we had a serious outage with our main finance system. Due to a failed backup script, the system was unavailable for two days. As a direct result of this, we had to send around 400 finance employees home for the period of the outage, as the system they used to do their jobs was not available.*

A rough calculation shows the direct costs of the outage:

- Lost productivity: 400 × 2 = 800 days
- Employee cost per day (fully loaded): $325
- Direct cost of outage (lost productivity): $260,000

Note: This is only a calculation of direct captured costs; it does not account for the extra work the finance team needed to do to catch up, etc.

Increased Toil

Without SRE's focus on eliminating manual and repetitive tasks through automation, operations becomes a steady stream of burdensome and time-consuming tasks for engineers. When people are doing work that could be done more quickly and accurately by machines—such as deploying and configuring new infrastructure and services, routine capacity planning, or log maintenance—this is valuable engineering time wasted on ad hoc and single-use solutions.

Reduced Velocity

People are valuable and machines are cheap. SRE recognizes this principle and encourages engineers to focus on creative tasks such as software design and, where necessary, handling outages. If engineers are continually being pulled into operational tasks, their valuable creativity is unavailable for new solutions in tooling, automation, and other system improvements. This slows launches and, in turn, feature velocity.

Retrofitting

The costs of reliability failures are typically more apparent when a service or system is in widespread use and has grown large and complex. As a result, organizations often become motivated to retrofit reliability once it has become unwieldy. This is possible, but it's considerably more complex and costly than building it in from the start. SRE's focus on reliability engineering seeks to avoid the compounding effects of early operational and architectural decisions before they become endemic and more difficult and disruptive to correct. Involving SRE in the early development and architectural decisions of critical services can reduce the long-term cost of implementing best practices.

Penalties

Most software as a service (SaaS) offerings require providers to compensate customers if they fail to meet their contractual commitments. The service level agreement (SLA) offered by Google Cloud Services is an example. This means there is often a direct financial impact if a customer-facing service is not meeting its SLAs.

Underneath the external SLAs that Google commits to are internal service level objectives (SLOs). These are measured targets of reliability that are set and adjusted according to business requirements. See Chapter 8 to learn how SLOs are designed, implemented, and used.

Cost to Manage

Readers with experience running and managing services will know about the overall cost of managing unavailability. The authors' collective experience indicates that downtime has a fanout cost in the range of 100× to 1000×—that is, 1 hour of downtime on a major system can incur a management cost of 100 to 1,000 hours. Table 2-1 gives an example breakdown.

Table 2-1. Example cost in hours to manage one hour of downtime

Management actions	Cost (engineer hours)
Operations team directly managing the outage	5–10
Post-incident review, timeline analysis, remediation actions (tracking bugs)	10–20
Internal communication, presentation and discussion of outage	20–50
Remediation of repair items	100
Customer communications	100

There are other follow-on costs in addition to those listed in the table. For instance, most companies will usually slow down software rollouts after major downtime, which impacts the product roadmap.

System Atrophy over Time

No application is an island, and the environment in which it operates does not stand still. Changes to services are required, and therefore outages and remediation are part of life and to be expected.

If we don't invest in and improve our systems, everything starts to atrophy over time. What was a small bug fix last year is now a major project, as the simple fix now requires updates in multiple other systems. SRE is deliberate in its identification and prioritization of these latent risks in the context of overall business goals.

In complex systems, when engineers try to make a change and it fails in unexpected ways, it can come as a shock. In the worst case, engineers start to become afraid of the applications. As a result, they don't make changes, and the whole problem gets worse. The system ends up in a downward spiral (see the following sidebar for insight from the authors' experience).

THE CUMULATIVE DAMAGE OF TECHNICAL DEBT

A SaaS startup had grown very quickly over a period of three or four years. It realized 200% year-on-year growth in revenue and prioritized growing the business over back-end hygiene, patching, etc.

But there came a point where the drip, drip, drip of downtime started to wear on customers. The startup began to get a reputation for being unreliable.

The fix turned out to be "all hands on deck" for 12 months as the whole operations team systematically worked through every system and caught up on basic hygiene. As a result of these efforts, global uptime improved from 99.84% to 99.999% (from one hour of downtime per month to seconds per month).

When engineering presented the plan to the CEO to approve the amount of scheduled maintenance needed to address the issues, the following conversation took place:

> *CEO: This looks like a reasonable plan; we finish at phase Z.*
> *Ops lead: Sorry, we ran out of letters in the alphabet to represent the phases of this get-well plan.*

System atrophy often manifests either as an increase in manual workarounds or a higher than expected level of downtime or performance degradation. SRE, with its focus on SLOs and reduction of toil, will generally look to improve or fix these situations over time, reducing costs and improving quality of service.

Patterns of SRE for Different Organizations

How much reliability do you need? SLOs and other measures of expected reliability are effective planning and operational tools only if this question has first been answered well. The answer is unique to your organization and will change over time with internal and external factors, but it will continue to be a measure of value: the benefit of building and maintaining reliable systems balanced against their cost.

There is no one-size-fits-all quantitative model for determining the value of reliability; no two organizations value business risk and operational costs exactly alike. But organizations in similar sectors and at similar stages of development tend to evaluate reliability in similar ways due to their exposure to similar internal and external market and technical forces.

Early-Stage and Smaller Organizations

Reliability should ideally be considered from the beginning of an organization's development and operational lifecycle and integrated early into its processes. It is far less costly and disruptive to expand an existing reliability structure later on than it is to introduce one that is not present.

Small and early-stage organizations often have limited resources, and they may defer reliability (and other concerns, such as security) to focus on fast and early development. The rationale is tempting: focus all efforts on getting a product to market and worry about reliability later. But it is exactly at this stage that small investments in SRE practices are least costly to implement and return the most value over time.

The following SRE practices are relatively simple and inexpensive to implement, yet they provide immediate improvements to system reliability and enable sustained velocity over time by keeping feature releases smooth, frequent, and predictable.

Automated unit and integration test coverage

The earlier a bug is detected, the less costly it is to fix and the less damage it can do. The cost of fixing a bug detected by automated tests on a developer's workstation can be near zero. Once that same bug reaches production, the resulting cost of outages, data corruption, and other issues it causes can be painfully high.

Automated continuous integration/continuous deployment pipeline

Making all the steps between writing and deploying software automatic, repeatable, and deterministic significantly reduces the risk of introducing a problematic change to production (see Chapter 18). Changes introduced to production represent the largest category of outage triggers at Google. Manual deployment to production (aside from being prohibitively time-consuming) is unacceptably error-prone. CI/CD generally encourages many of the best practices discussed in this section by enforcing them or making them available by default, and making it simple to integrate them as part of all software development and deployment.

Observability

Quality monitoring of a service can be relatively expensive to implement, but it is necessary to remain aware of problems your users are experiencing and to act effectively during an outage. Having good instrumentation across your service is important for debug-

ging and troubleshooting, but the goal isn't to have engineers continually looking at dashboards. Let the machines watch the service and alert a valuable engineer only when there's a problem. Choose alerts that are actionable and that reflect a poor user experience, so that when an engineer is alerted, their time is well spent. Good monitoring paired with well-chosen indicators of service reliability are the foundation for SLOs, allowing informed decisions about where to invest engineering resources. See Chapter 9.

Incident response process

Maintaining an effective incident response system within your organization is a best practice that clearly demonstrates its worth. The cost of an outage grows with every minute, so a simple and practiced system that reduces the length and severity of outages will return immediate measurable value to the organization. The response system doesn't have to be complex. Any system that has clear responder roles, mitigates outages effectively, and captures critical information about the outage (enabling engineering to prevent a repeat of the same failure mode) is sufficient. A blameless postmortem culture that drives incremental engineering improvements after the outage is resolved is also vital. See Chapter 10.

Design review

A design review focusing on reliability doesn't have to be an onerous process that slows releases. The goal is in fact the opposite: catch problems early while they are quick and inexpensive to correct, in order to maintain release velocity over time. The process can be as simple as spending a few minutes during a regular engineering meeting analyzing and discussing proposed service changes, specifically looking for potential reliability risks. Taking time to consider best practices such as the separation of failure domains, redundancy, rate limiting, rollbacks, and data recovery at this stage addresses common sources of early failure and technical debt, and it returns that investment many times over through more consistent release velocity and fewer, less severe outages.

Growing and Mature Organizations

All of the benefits to small and early-stage organizations that we just covered also accrue to more established ones, where sustained scale and efficient resource management become more important. The following are some reasons to continue to develop SRE within your organization as it grows.

Strategic engineering decisions

SLOs are more than targets representing the user's reliability expectations for a service: they are a communication tool. SLOs represent a common agreement between different parts of the organization with different incentives, and so are key to making strategic decisions about how the organization should invest engineering time and resources. For a mature organization with many services, SLOs provide the necessary data to prioritize work effectively, ensuring engineers focus on the most important characteristics of the most business-critical services. Some services warrant the higher cost of increased reliability, while others can serve their purpose adequately at a less costly level of reliability. This deliberate allocation of resources optimizes your engineering budget, preventing wasted effort on less impactful areas, and allows the organization to grow beyond the limitations imposed by a reactive approach to outages and engineering.

Managing complexity

Mature systems accumulate complexity over time, and with it an increased risk of outages due to latent bugs, obsolete dependencies, and stale configurations. These issues are certainly capable of causing spectacular outages, but complexity also tends to increase and magnify the small problems that—when compounded—strain engineering resources and drive up operational costs. SRE actively works to prevent and reduce this damaging technical debt through standardization and automation, making systems more manageable, easier to reason about, and ultimately less expensive to operate and maintain.

Incident response

An efficient and effective incident response offers particular value to large and mature organizations. The most obvious reason is that these organizations typically process larger and higher-value transactions, so the per-second cost of an outage is simply higher. Established organizations also are generally more in the media spotlight and held to a higher standard of reliability than a startup, so the bad press surrounding an outage tends to be amplified. Probably the most underappreciated benefit of effective incident response to large organizations is the smooth handling of outages, enabled by clearly defined roles. With size and scale comes complexity. An unorganized response marked by ad hoc action and communication easily becomes chaotic and impedes service restoration. See Chapter 10.

Centrally maintained tools

In larger organizations (like Google), SRE automation may evolve into the development of centralized infrastructure, or platforms, that provide standard, reliable solutions for common infrastructure needs across many or all of the organization's engineering teams. These allow development teams to focus on their respective business solutions, increasing overall organizational efficiency. This standardization also improves incident response, as responders across the organization are already familiar with the relevant infrastructure—which may not be the case when different areas of the organization maintain different tooling.

What to Consider When Adopting SRE

Large or small, every organization's approach and journey to SRE will be different. Everyone starts from a different place, and people have different priorities and desired end points. In light of this, there are a few key questions to ask yourself as you plan your SRE journey:

What are your biggest operational challenges?

> Adopting SRE (or any other practice) as a "hammer" in hopes of resolving a large number of issues without first understanding or at least identifying them is unlikely to be successful. Take time to identify a few top culprits over the previous quarter or two. Frequent or severe outages? Deployments that are slow to roll out or need to be rolled back? Unaddressed backlogs due to high toil? Low motivation or burnout? Practices that are unsustainable at scale? Not meeting customer expectations for reliability and performance? The following chapters describe best practices to help address these issues.

What are your available resources?

> Having engineers with a blend of software engineering and operational skills is a foundation of SRE. If you don't have them already, are you prepared to hire and/or train them? Do you have a culture that will revert back to reflexive quick fixes when pressure mounts, or are you willing to dedicate the required engineering time and resources to build tooling and automation and address the causes of outages identified by postmortems?

Can you start small?

It isn't necessary to convert your entire organization to SRE practices at once. Implementing basic practices such as SLOs, actionable alerting, and toil reduction on even a single critical service—or one that would benefit most—can serve as a proving ground to gain greater traction within your organization.

What is at stake?

To establish risk, you must quantify the business cost of poor service reliability. Every service, whether streaming video, providing retail transactions, or supporting medical operations, has users who expect a certain level of reliability from it. Understanding and quantifying those expectations, and what to do when the service falls short or exceeds them, is the subject of Chapter 8.

What is the nature of your user interactions?

Is a user interaction such as a query or a purchase permanently lost if the system is unavailable? Or will the user or downstream client simply try again? Very short outages are usually far less damaging than longer ones, even if they are relatively frequent. This is because most users will just try again (though this isn't always true, for example in the case of impulse buys or flash sales). Retry attempts tend to decrease as the length of an outage increases, so the cost of an outage grows over time.

In situations where demand is very high and supply is very low, such as concert ticket sales and "door-crasher" sales, there is likely sufficient demand to ensure that all items are eventually sold despite service outages. While there may be no immediate financial loss to the business, the reputational damage due to upset customers who feel they were denied an opportunity can be significant.

Is timing a factor?

For organizations that have annual, monthly, or even daily business-critical periods, the value of reliability increases dramatically at these times. This is when failure has the highest financial or reputational impact. Keeping critical services running without issues during these periods is paramount for the organization. In the worst case, should the service fail, restoring it as quickly as possible becomes the top priority.

Conclusion

The principles and practices of Site Reliability Engineering can offer measurable benefit to organizations of different sizes and stages of growth. Adopting SRE practices can translate directly to improved financial performance by minimizing the costly consequences of system downtime, but the extended business advantages are less obvious. By applying engineering principles to operations, SRE enables services to scale efficiently, reducing the need for constant and costly manual intervention and thus freeing up valuable engineering time. This allows technical teams to focus on innovative solutions delivered on a predictable schedule, rather than being consumed by fixes and outages. SRE should be viewed as a business investment—one that optimizes operational spending and strengthens the foundation of operations and infrastructure—to support the strategic goals of the organization.

1 Dustin Smith, "2021 Accelerate State of DevOps Report Addresses Burnout, Team Performance", Google Cloud blog (2021).

2 Incident alerts at Google today are received on an engineer's mobile device, but originally they went to physical pagers, and the name has stuck.

Previous chapter
1. Introduction: Modeling What We Do and Why

Next chapter
3. Trends in Production